SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: 1st November 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	VBall
Project / Solution Name	SMErgers
Name (Leader)	Vincent Chew Wen Jie
Email & Phone (Leader)	vinchew00@gmail.com | +6011-20208512
University Affiliation(s)	Universiti Sains Malaysia
Live Prototype URL	https://sm-ergers-1.replit.app
Code Repository URL (public / access granted to judges)	https://github.com/Anonys-sky/SMErgers

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

Messages enter through a webhook using Meta's WhatsApp Business Cloud API format or through imported WhatsApp chat exports. For the live demo, an unofficial WhatsApp Web connector reads a dedicated test group and logs incoming messages.

The backend extracts scope changes, quantities, prices and agreement status using the signed quote, rate card, company directory and past jobs. Python rules process messages first. When selected, an AI reader sends chat text to our RunPod endpoint running Llama 3.1 8B Instruct. Deterministic checks validate extracted prices and statuses; the rules reader remains available if AI inference fails.

Changes enter a server-enforced approval workflow involving the project manager, project director and client, with configurable business approval limits. Each confirmation creates a quote version, queues a group update and records a hash-chained activity log entry.

Outputs include a draft bill, CSV export, read-only JSON API, activity log and benefit report. Flask serves the web interface, hosted on Replit. Prototype data is stored in memory and resets when the application restarts.

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
Python 3.12	Backend application language	PSF License
Flask 3.1.3	Web framework and application routes	BSD-3-Clause
Jinja2, MarkupSafe, Werkzeug (installed with Flask)	HTML templates, safe escaping of chat text and request handling	BSD-3-Clause
tzdata 2026.5	Time-zone handling for message timestamps	Apache-2.0

Name & Version	Purpose in Project	License
pytest 9.1.1	Automated testing	MIT
Pygments 2.21.0	Test-output syntax highlighting	BSD-2-Clause
vLLM — [INSERT VERSION]	Serves Llama 3.1 8B Instruct on RunPod	Apache-2.0
Ollama — [INSERT VERSION]	Local model testing	MIT
Node.js ≥22.12	Runtime for the optional local WhatsApp test connector	MIT; bundled components have their own notices
whatsapp-web.js ^1.34.7	Unofficial connector for reading a dedicated WhatsApp test group	Apache-2.0
Puppeteer — [INSERT VERSION]	Browser automation used by the WhatsApp test connector	Apache-2.0
qrcode ^1.5.4	QR code generation for the connector's pairing page	MIT

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
RunPod Serverless API (OpenAI-compatible)	GPU inference for Llama 3.1 8B Instruct using a vLLM worker	Pay per second, from the $25 Sponsor Credit
WhatsApp Business Cloud API	Design and webhook format for receiving group messages and delivering updates. Not called live in the prototype: messages are synthetic and outgoing messages are queued and shown on screen.	Not used live; production use needs a verified business account
WhatsApp Web, through the unofficial whatsapp-web.js library	Optional local test connector: reads a synthetic test group directly on a spare test number. Not an official API and not part of the product	None; may breach WhatsApp's terms, so test numbers only
None Other	No hosted GenAI API, maps, payments or data feeds are called	N/A

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
Replit	Development environment, hosting and deployment of the live Flask prototype	Replit Core, from the preliminary round $20 sponsor credit
RunPod (Serverless) GPU inference for the AI reader: Llama 3.1	RunPod (Serverless) GPU inference for the AI reader: Llama 3.1	RunPod (Serverless) GPU inference for the AI reader: Llama 3.1

Service & Provider	Purpose in Project	Plan / Region
GitHub	Public source repository and commit history	GitHub Free; public repository

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
Llama 3.1 8B Instruct	Meta; hosted through vLLM on our RunPod endpoint, with Ollama used for local testing	Pre-trained model used for inference only. Extracts scope changes from chat text. No training or fine-tuning performed by the team.	Llama 3.1 Community License
The model supports extraction only. Deterministic code validates prices and statuses, and people control approvals and billing. A second extraction pass checks for missed changes, with the same validation applied to additional results. The rules reader is used if the model is unavailable or does not respond within 60 seconds. Development assistants listed in Section 4.3 do not run inside the application.			

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
Synthetic event chats (8, three of them blind test chats written by separate AI agents), quotes (7), vendor invoice, answer keys	Written by the team, in data/ of the repository	synthetic	Apache-2.0	N
Synthetic job history covering 10 past jobs	Written by the team, in data/history/past_jobs.json	synthetic	Apache-2.0	N

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
Synthetic company directory	Written by the team, in data/team.json	synthetic	Apache-2.0	N
Dedicated WhatsApp demo-group messages	Team-controlled test group, read through the optional local connector	Self-collected	Team-controlled test content; not publicly licensed	Y — may include participants' names, phone numbers or account identifiers
Event-team survey with seven responses	Team's Google Form; aggregated findings in docs/survey.md	Self-collected	Not publicly licensed; only aggregated findings published	Y — raw responses include names and companies; raw sheet kept outside the application and repository

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☑ Yes ☐ No

If yes, list them below:

Tool & Version	**Used For**	**Extent of Use (minor / moderate / major)**
Claude (Claude Code; Opus 5.5 and Sonnet 5.5), Anthropic	Writing and refactoring code, tests, documentation, the pitch deck and the proposal text; research and analysis	Major
Replit Agent — Replit	Run configuration, deployment publishing, startup scripts and related tests	Minor
ChatGPT (Codex; GPT-6.1 Sol, GPT-6.1 Terra, GPT-6.1 Luna)	Writing and refactoring code, tests, documentation, the pitch deck and the proposal text; research and analysis	Major

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

The problem comes from a teammate's experience at an event company. The team selected the problem and made the design decisions. We reviewed, edited and tested AI-generated code through small commits and 357 automated tests. We manually checked the running application as each role. Both readers were scored against answer keys committed before scoring, with successful and unsuccessful runs recorded in docs/results.md. Team members reviewed how messages become changes, how prices are selected, how approvals are enforced and how the activity log detects edits. The team takes responsibility for the submitted implementation and claims.

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☑ Yes ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
SDC / XMUM AI Club (platform partner: Replit)	Replit Core subscription for each participant	Development environment and hosting of the live prototype
SDC / RunPod	US$25 team credit	Used: Serverless vLLM endpoint 2y90x5ojwpgxnh (RTX 4090, 24 GB) runs the AI reader's model, Llama 3.1 8B Instruct.

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

Replit hosts the submitted prototype and provides the development environment. The application uses Python and Flask, so it could run on another Python-compatible host with deployment configuration changes. RunPod provides GPU inference for the AI reader. It could be replaced by another host serving a compatible open model. The application also includes a rules reader that works without GPU inference, although its extraction performance differs. No model training or fine-tuning was performed. KrackedDevs resources were not used.

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

The prototype processes WhatsApp message text, sender names, timestamps and identifiers; phone numbers or other personal details may appear in chats. Demo fixtures are synthetic. Project records are held in server memory and cleared on restart or reset, while the optional WhatsApp connector saves linked-account session data on disk. Uploaded chats are processed on the application host. When the RunPod AI reader is selected, sender names, chat text and relevant quote details are sent to our endpoint without automatic anonymisation. API keys are kept in Replit Secrets. A separate survey contains names and companies; only aggregated results are published, with raw responses outside the application and repository. Planned protections include authenticated project access, identifier masking before AI requests, restricted and encrypted persistent storage, retention limits, deletion controls and removal of WhatsApp sessions after testing. These safeguards are not yet fully implemented; external-service retention remains unverified.

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

The AI reader can miss or misinterpret changes and treat an unconfirmed change as ready. On four blind synthetic chats, it correctly extracted 27 of 36 changes, compared with eight for the rules reader. After fixes, it extracted 33 of 36 on the same chats; this is not an independent evaluation. The samples are small and synthetic, so we claim no real-world accuracy.

Deterministic checks reject unsupported prices. Billing requires human review and client confirmation; missing prices and discounts require manual handling. If AI inference fails or times out, the rules reader remains available.

The prototype stores data in memory and simulates sign-in. Official WhatsApp group integration has eligibility and group-size constraints. The unofficial demo connector may breach WhatsApp's terms and is not part of the intended production integration.

7. Compliance & Readiness Checklist

Tick all boxes that apply before submitting.

- ☑ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☑ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☑ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- ☑ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☑ Any use of official SDC sponsor resources has been declared in Section 5.
- ☑ The prototype described here is accessible to judges through the links provided in Section 1.
- ☑ All fields in this template have been completed within the word limits stated.

8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.

[Signature]



[Date]

31th October 2026